



Universidade do Minho
Escola de Engenharia
Licenciatura em Engenharia Informática

Computação Gráfica

Trabalho Prático

Grupo 15

Edgar Ferreira	Fernando Pires	Sara Silva	Zita Duarte
a99890	a77399	a104608	a104268

Data da Receção	
Responsável	
Avaliação	
Observações	

Computação Gráfica

Edgar Ferreira

a99890

Fernando Pires

a77399

Sara Silva

a104608

Zita Duarte

a104268

março, 2025

Índice

1. Introdução	1
2. Generator	2
2.1. Funcionamento do Generator	2
2.2. Formato do ficheiro .3d	2
2.3. Organização e orientação dos triângulos	3
2.4. Convenção de Nomenclatura	3
2.5. Primitivas	3
2.5.1. Plano	4
2.5.1.1. Centragem e divisão do espaço	4
2.5.1.2. Cálculo das coordenadas de cada quadrado	4
2.5.1.3. Exemplo visual	5
2.5.2. Caixa	5
2.5.3. Esfera	6
2.5.3.1. Cálculo das Coordenadas	7
2.5.3.2. Conversão para Coordenadas Cartesianas	7
2.5.3.3. Triangulação da Superfície	8
2.5.4. Cone	9
2.5.4.1. Cálculo da Geometria	9
2.5.4.2. Coordenadas dos Pontos	9
2.5.4.3. Processo de Geração	10
2.5.5. Torus	10
2.5.5.1. Cálculo da Geometria	10
2.5.5.2. Coordenadas dos Pontos	11
2.5.5.3. Processo de Geração	11
2.5.6. Helix	11
2.5.6.1. Cálculo da Geometria	12
2.5.6.2. Coordenadas dos Pontos	12
2.5.6.3. Geração da Superfície	12
3. Engine	13
3.1. Renderização da Cena	13
3.1.1. Leitura e Interpretação do XML	13
3.1.1.1. Configuration	13
3.1.1.2. Window	14
3.1.1.3. Camera	14
3.1.2. Renderização dos Modelos	14
3.1.2.1. Leitura de ficheiros e <i>parsing</i>	14
3.1.2.2. Armazenamento e desenho	15
3.2. Exploração da Cena	15
3.2.1. Movimento Orbital	15
3.2.2. Zoom	16
3.2.3. Outros	16
4. Utilitários	17
4.1. Bibliotecas	17
4.2. Scripts	17

4.3. Classes Comuns	18
5. Conclusões e Trabalho Futuro	19

Lista de Figuras

Figura 1	Ilustração de uma das divisões do plano	5
Figura 2	Plano com 4 de largura e 6 divisões	5
Figura 3	Caixa com 2 de largura e 3 divisões	6
Figura 4	Ilustração da divisão da esfera	7
Figura 5	Ilustração de uma esfera com 1 de raio, 10 <i>slices</i> e 10 <i>stacks</i>	8
Figura 6	Ilustração de uma <i>slice</i> e uma <i>stack</i> de um cone	9
Figura 7	Ilustração de um cone com raio 1, altura 2, 4 <i>slices</i> e 3 <i>stacks</i>	10
Figura 8	Ilustração de um torus com raio maior de 3, raio menor de 1, 20 <i>sides</i> e 15 <i>rings</i>	11
Figura 9	Ilustração de uma hélice com raio 2, altura 5, 10 <i>turns</i> e 30 <i>slices</i>	12
Figura 10	Desenho de um ficheiro porsche.obj	14
Figura 11	Desenho de um ficheiro ferrari.obj	15
Figura 12	Desenho de um ficheiro spider.obj	15
Figura 13	Diagrama do movimento orbital da câmara	16

1. Introdução

Este relatório foi realizado no âmbito da Unidade Curricular de Computação Gráfica (CG) no ano letivo 2024/2025 e descreve o desenvolvimento da primeira fase do projeto, que consiste na implementação de duas componentes essenciais, o **Generator** e o **Engine**, para geração e visualização das primitivas gráficas, respetivamente.

2. Generator

Como parte integrante deste trabalho prático, foi desenvolvido um **Generator**, responsável pela criação de primitivas gráficas tridimensionais com base em parâmetros fornecidos pelo utilizador. Este programa tem como objetivo gerar modelos geométricos compostos por triângulos, garantindo que cada primitiva possa posteriormente ser utilizada no processo de renderização.

2.1. Funcionamento do Generator

O programa recebe como argumentos as características específicas da primitiva pretendida, tais como o seu tamanho, número de divisões ou outras propriedades geométricas relevantes. Com base nestes parâmetros, é gerado um conjunto de pontos que define as coordenadas tridimensionais dos vértices necessários para formar a primitiva.

Todas as primitivas são representadas exclusivamente através de triângulos. Este formato foi escolhido tanto pela simplicidade de representação como pela compatibilidade com o pipeline gráfico do OpenGL, que utiliza triângulos como unidade base para renderização. Após a geração dos pontos e respetivos triângulos, estes são armazenados num ficheiro com a extensão `.3d`, utilizando uma estrutura de dados padronizada criada pelo grupo.

2.2. Formato do ficheiro `.3d`

Os ficheiros `.3d` produzidos seguem um formato textual simples e estruturado. A primeira linha contém o número total de vértices do modelo (n), seguido de um conjunto de n linhas, cada uma representando um vértice tridimensional. Cada linha de vértice apresenta as suas coordenadas espaciais separadas por espaço, seguindo a ordem $x\ y\ z$.

Adicionalmente, cada grupo de três vértices consecutivos define um triângulo da primitiva. Esta organização garante que a primitiva é definida integralmente por uma coleção de triângulos, assegurando a compatibilidade com mecanismos como o `glDrawArrays` em OpenGL.

Exemplo de conteúdo de um ficheiro `.3d` para um plano simples:

```
6
-1 1 0
-1 -1 0
1 1 0
1 -1 0
-1 -1 0
1 -1 0
```

Este exemplo define dois triângulos que formam um plano completo, adequado para cobrir o ecrã, assumindo que a câmara está centrada na origem e sem transformações aplicadas.

2.3. Organização e orientação dos triângulos

Durante a geração de cada primitiva, é assegurada a orientação correta dos triângulos, de forma a garantir que as faces visíveis da primitiva estejam corretamente orientadas para o exterior. Para tal, os vértices de cada triângulo são organizados no sentido **anti-horário**, de acordo com a convenção de *face culling* utilizada pelo OpenGL. Esta prática é essencial para permitir a ocultação automática das faces traseiras (*back-face culling*), otimizando o processo de renderização e evitando artefactos visuais.

2.4. Convenção de Nomenclatura

Os ficheiros produzidos seguem uma convenção de nomenclatura definida pelo utilizador no momento da execução do programa. O nome do ficheiro é passado como argumento final da linha de comando. Por exemplo, ao executar o comando:

```
generator cone 1 2 4 3 cone_1_2_4_3.3d
```

o ficheiro resultante terá o nome `cone_1_2_4_3.3d`, permitindo que o utilizador personalize a nomenclatura de acordo com os parâmetros usados ou com qualquer outra convenção desejada. Esta flexibilidade facilita a organização e gestão dos diferentes modelos gerados.

2.5. Primitivas

No decorrer deste trabalho, foi desenvolvido um conjunto de primitivas gráficas que podem ser geradas através do *Generator*. Inicialmente, foram implementadas as seguintes primitivas básicas:

- Plano
- Caixa
- Esfera
- Cone

Cada uma destas primitivas possui um conjunto específico de parâmetros que definem as suas dimensões e subdivisões. Estes parâmetros são fornecidos pelo utilizador no momento da execução do programa.

Além das primitivas base, o grupo decidiu ir mais além e implementar duas primitivas adicionais, como forma de desafio e enriquecimento do projeto:

- Torus
- Helix

A forma como cada primitiva pode ser gerada está descrita na mensagem de ajuda do programa, conforme o seguinte formato:

```
Usage: generator <command> <args> <output file>
```

Commands:

```
generator plane <length> <divisions> <output file>
generator sphere <radius> <slices> <stacks> <output file>
generator cone <radius> <height> <slices> <stacks> <output file>
generator box <length> <divisions> <output file>
generator torus <innerRadius> <outerRadius> <slices> <stacks> <output file>
generator helix <radius> <height> <coils> <slicesPerCoil> <output file>
```


Com esta abordagem, o *Generator* permite a criação flexível de seis primitivas diferentes, cobrindo tanto formas básicas como objetos de maior complexidade geométrica. A implementação de figuras como o `torus` e a `helix` trouxe desafios adicionais relacionados com parametrização e geração de coordenadas, contribuindo para o desenvolvimento de uma maior compreensão sobre a construção de modelos 3D complexos.

Segue-se a descrição detalhada do processo de triangulação e geração de cada uma destas figuras.

2.5.1. Plano

Para a correta geração do plano, é necessário definir dois parâmetros fundamentais:

- **Tamanho:** a largura total do plano (`float`);
- **Divisões:** o número de subdivisões em cada eixo (inteiro).

2.5.1.1. Centragem e divisão do espaço

De modo a garantir que o plano fica centrado na origem (0,0,0), a largura é dividida por 2, obtendo-se o valor `half`, que representa a distância entre a origem e uma das extremidades do plano.

O plano é dividido num conjunto de quadrados, formando uma grelha. Cada quadrado é por sua vez dividido em dois triângulos, para facilitar a renderização através de primitivas triangulares. Para calcular o espaçamento entre os pontos da grelha, usamos:

$$\text{step} = \frac{\text{tamanho}}{\text{divisoes}}$$

Com este `step`, conseguimos calcular as coordenadas de cada vértice de uma subdivisão.

2.5.1.2. Cálculo das coordenadas de cada quadrado

Cada subdivisão (quadrado) é definida por 4 pontos:

- P1, P2, P3, P4

Estes pontos são obtidos da seguinte forma:

$$x1 = -\text{half} + j * \text{step}$$

$$z1 = -\text{half} + i * \text{step}$$

$$x2 = x1 + \text{step}$$

$$z2 = z1 + \text{step}$$

Com `y=0`, dado que o plano está na horizontal.

Com estes pontos, formamos 2 triângulos por cada quadrado:

- **Triângulo 1:** P1, P2, P3
- **Triângulo 2:** P1, P3, P4

Para compatibilidade com representações de *double-sided geometry*, foi também gerada uma face invertida no sentido de `y -`.

2.5.1.3. Exemplo visual

A imagem seguinte ilustra a estrutura básica de uma subdivisão do plano:

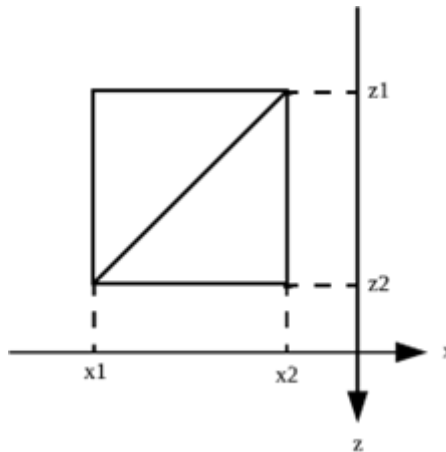


Figura 1: Ilustração de uma das divisões do plano

Por fim, uma visualização completa do plano com 4 unidades de largura e 6 divisões pode ser observada na figura abaixo:

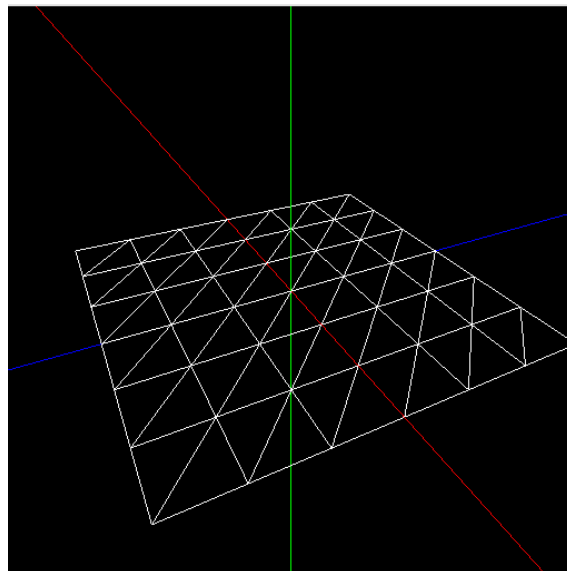


Figura 2: Plano com 4 de largura e 6 divisões

2.5.2. Caixa

Para a geração da caixa, optou-se por um método semelhante ao utilizado para o plano, mas adaptado para as seis faces do cubo. A caixa está centrada na origem, sendo definida por uma largura e um número de divisões, que permite controlar o nível de detalhe da sua malha.

Tal como no plano, começamos por calcular metade da largura (`half`) e o espaçamento entre divisões (`step`), com a seguinte fórmula:

$$\text{half} = \frac{\text{length}}{2}$$
$$\text{step} = \frac{\text{length}}{\text{divisions}}$$

Com estes valores, foi possível calcular as coordenadas x , y e z de cada vértice das faces. Para cada face, os vértices são calculados de forma semelhante ao plano, mas distribuídos de acordo com a orientação específica da face:

- **Base inferior** (face $y = -\text{half}$): Os pontos são definidos no plano horizontal, com y fixo em $-\text{half}$.
- **Base superior** (face $y = \text{half}$): Semelhante à base inferior, mas com y fixo em half .
- **Faces paralelas ao plano XY** ($z = -\text{half}$ e $z = \text{half}$): Os pontos são distribuídos ao longo do eixo x e y , mantendo z fixo.
- **Faces paralelas ao plano YZ** ($x = -\text{half}$ e $x = \text{half}$): Os pontos são distribuídos ao longo do eixo z e y , mantendo x fixo.

Cada face é construída com dois triângulos por célula da grelha, de forma a garantir que a superfície da caixa é completamente triangulada. A inserção dos pontos no vetor é feita de forma ordenada, de modo a garantir que cada face é corretamente descrita no ficheiro de saída.

Com este método, é possível gerar caixas de qualquer tamanho e com qualquer quantidade de divisões, permitindo um controlo flexível sobre o nível de detalhe e a complexidade da geometria gerada.

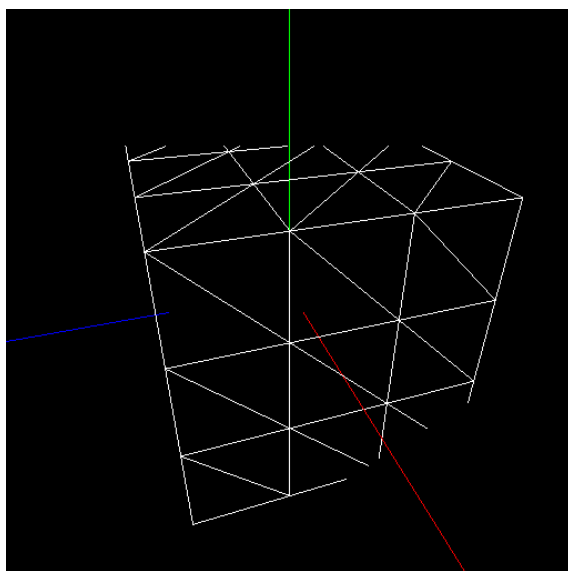


Figura 3: Caixa com 2 de largura e 3 divisões

2.5.3. Esfera

Para a construção da esfera, é necessário definir três parâmetros fundamentais: o raio, o número de *slices* e o número de *stacks*. Estes parâmetros controlam a precisão e a segmentação da malha esférica, permitindo ajustar o nível de detalhe da figura final.

A abordagem escolhida para a construção da esfera passa por dividir a sua superfície em secções verticais e horizontais. As *slices* representam divisões verticais (fatias longitudinais), enquanto as *stacks* representam divisões horizontais (fatias latitudinais). Desta forma, cada *slice* é subdividida em múltiplas *stacks*, resultando numa grelha de quadriláteros (ou faces), que é posteriormente triangulada para facilitar a renderização.

A Figura 6 ilustra esta divisão da esfera:

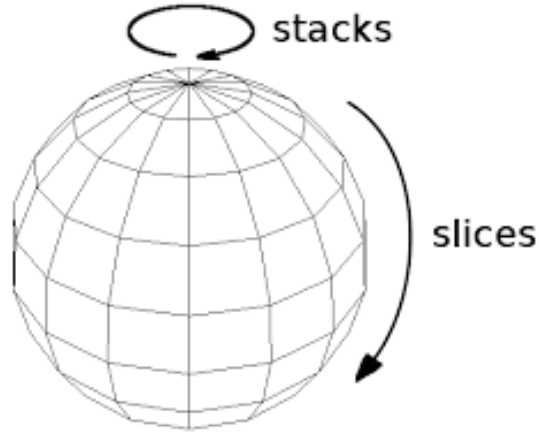


Figura 4: Ilustração da divisão da esfera

2.5.3.1. Cálculo das Coordenadas

Para calcular as coordenadas de cada vértice da malha, é necessário percorrer todas as combinações de *slices* e *stacks*, construindo secções adjacentes da superfície. A cada iteração, são determinados dois ângulos verticais (θ_1 e θ_2), e dois ângulos horizontais (φ_1 e φ_2), que definem as extremidades de cada face. Estes ângulos são calculados da seguinte forma:

- **Passo vertical entre *stacks*:**

$$\Delta \theta = \frac{\pi}{\text{stacks}}$$

- **Passo horizontal entre *slices*:**

$$\Delta \varphi = \frac{2 * \pi}{\text{slices}}$$

Para cada par de *slices* e *stacks*, os ângulos são:

$$\theta_1 = i * \Delta \theta$$

$$\theta_2 = (i + 1) * \Delta \theta$$

$$\varphi_1 = j * \Delta \varphi$$

$$\varphi_2 = (j + 1) * \Delta \varphi$$

2.5.3.2. Conversão para Coordenadas Cartesianas

Os pontos são definidos diretamente em coordenadas cartesianas através das seguintes fórmulas:

Para cada ponto:

$$x = r \sin(\theta) \sin(\phi)$$

$$y = r \cos(\theta)$$

$$z = r \sin(\theta) \cos(\phi)$$

Estas fórmulas são usadas para calcular quatro pontos em cada célula da grelha: dois pontos pertencentes à *slice* atual e dois pontos pertencentes à *slice* seguinte.

2.5.3.3. Triangulação da Superfície

Após calcular as coordenadas cartesianas dos quatro pontos que formam cada face quadrangular, esta é dividida em dois triângulos, garantindo compatibilidade com os formatos gráficos mais comuns. A triangulação é feita através da ligação direta dos vértices, segundo a ordem:

- **Triângulo 1:** P1, P4, P2
- **Triângulo 2:** P1, P3, P4

Isto assegura que a superfície da esfera é representada inteiramente por triângulos, simplificando a renderização.

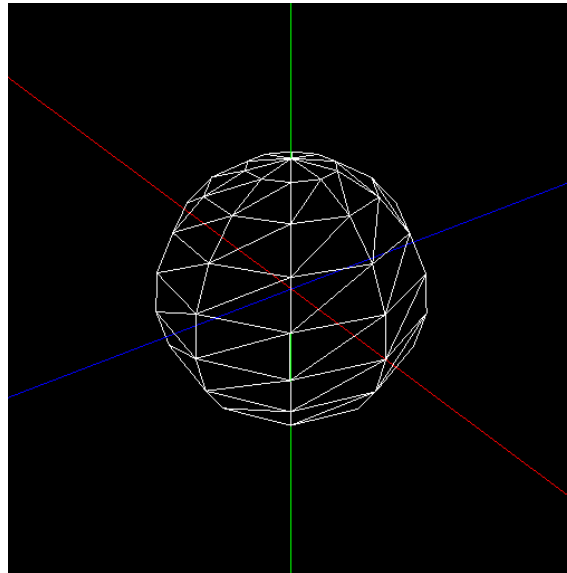


Figura 5: Ilustração de uma esfera com 1 de raio, 10 *slices* e 10 *stacks*

2.5.4. Cone

Para a geração do cone, são necessários os seguintes parâmetros:

- Um raio da base;
- Uma altura do cone;
- Número de *slices* (divisões radiais da base): dividem a base circular do cone em fatias iguais, correspondendo às subdivisões angulares da circunferência;
- Número de *stacks* (divisões verticais ao longo da altura): dividem a altura do cone em segmentos horizontais, formando uma sequência de anéis cada vez mais pequenos até atingir o vértice.

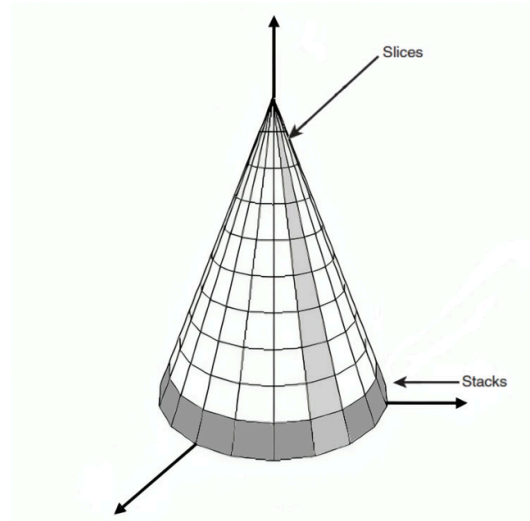


Figura 6: Ilustração de uma *slice* e uma *stack* de um cone

2.5.4.1. Cálculo da Geometria

A altura de cada *stack* e o ângulo de cada *slice* são calculados da seguinte forma:

- Altura de cada *stack*

$$h = \frac{\text{altura total}}{\text{n}^{\circ} \text{ de stacks}}$$

- Ângulo de cada *slice*

$$\alpha = \frac{2\pi}{\text{n}^{\circ} \text{ de slices}}$$

2.5.4.2. Coordenadas dos Pontos

Em cada *stack* e *slice*, são calculadas as coordenadas dos pontos que definem os triângulos. Para cada *stack* j , a altura é:

$$y = j * h$$

O raio correspondente à *stack* é:

$$\text{raio stack} = \text{raio} * \left(1 - \frac{j}{\text{stacks}}\right)$$

Os pontos laterais são calculados com base no ângulo de cada *slice*:

$$x = \text{raio stack} * \sin(\theta)$$

$$z = \text{raio stack} * \cos(\theta)$$

2.5.4.3. Processo de Geração

O processo de geração é feito com dois ciclos aninhados:

- Um ciclo exterior que percorre cada *slice*;
- Um ciclo interior que percorre cada *stack*.

Para cada fatia e cada *stack*, são gerados dois triângulos que formam um quadrilátero da malha. Na prática, o cone é dividido em *slices* verticais e cada *slice* é subdividida em *stacks* horizontais.

Base do Cone

Adicionalmente, a base é um disco circular definido por triângulos, todos com um vértice no centro da base e os outros dois em pontos consecutivos da circunferência.

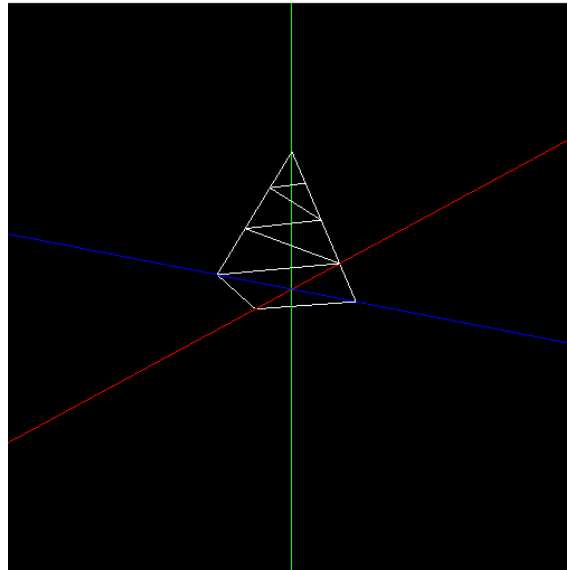


Figura 7: Ilustração de um cone com raio 1, altura 2, 4 *slices* e 3 *stacks*

2.5.5. Torus

Para a geração de um torus, são necessários os seguintes parâmetros:

- **Raio maior** (*major radius*), que define a distância do centro do torus até ao centro do “tubo”;
- **Raio menor** (*minor radius*), que define o raio do “tubo” propriamente dito;
- **Número de rings**: dividem a secção circular maior (a forma de donut) em segmentos;
- **Número de sides**: dividem a secção circular menor (a forma do tubo) em segmentos.

Cada par de rings consecutivas é unido por uma faixa de quadriláteros, e cada quadrilátero é dividido em dois triângulos, formando a malha do torus.

2.5.5.1. Cálculo da Geometria

A posição de cada vértice é definida em coordenadas cartesianas utilizando ângulos polares e cilíndricos. Cada ponto é calculado com duas rotações:

- Rotações em torno do eixo central (definidas pelo ângulo θ);
- Rotações em torno do círculo menor (definidas pelo ângulo ϕ).

Para cada ring i , o ângulo é:

$$\theta = \frac{2\pi i}{\text{rings}}$$

Para cada side j , o ângulo é:

$$\varphi = \frac{2\pi j}{sides}$$

2.5.5.2. Coordenadas dos Pontos

Com cada par de ângulos, as coordenadas são calculadas:

- Para cada ponto na superfície do torus:

$$x = (\text{raio maior} + \text{raio menor} \cos(\varphi)) \cos(\theta)$$

$$y = \text{raio menor} \sin(\varphi)$$

$$z = (\text{raio maior} + \text{raio menor} \cos(\varphi)) \sin(\theta)$$

2.5.5.3. Processo de Geração

A geração do torus é feita com dois ciclos aninhados:

- Um ciclo exterior para percorrer cada *ring* (divisão maior);
- Um ciclo interior para percorrer cada *side* (divisão do tubo).

Para cada par de *rings* consecutivas e cada par de *sides* consecutivas, são definidos dois triângulos que formam um quadrilátero da malha.

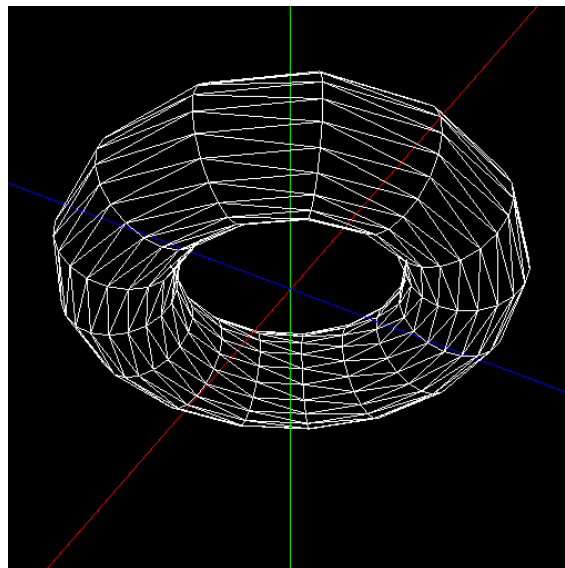


Figura 8: Ilustração de um torus com raio maior de 3, raio menor de 1, 20 *sides* e 15 *rings*

2.5.6. Helix

A geração da hélice baseia-se em 4 parâmetros principais:

- Raio da hélice, que define a distância da hélice ao eixo central (o quão “larga” é a mola);
- Altura da hélice, que define o comprimento total da mola;
- Número de voltas (*turns*), que indica quantas rotações completas a hélice dá ao longo da sua altura;
- Número de *slices* por volta, que define quantas secções cada volta é subdividida, controlando a suavidade da curva.

A hélice é uma curva helicoidal, que sobe em espiral ao longo do eixo vertical (eixo Y). A espiral é aproximada por pequenos segmentos de reta, definidos por cada *slice*.

Cada *slice* corresponde a um pequeno avanço angular (em torno do eixo central) e a um pequeno avanço vertical (ao longo da altura). A altura total é dividida por (`turns * slices`), garantindo uma distribuição uniforme.

2.5.6.1. Cálculo da Geometria

A posição de cada ponto da hélice é determinada em coordenadas polares, onde o ângulo (em torno do eixo Y) cresce com cada *slice*, e a altura cresce proporcionalmente.

- O ângulo de cada *slice* é:

$$\theta = \frac{2\pi}{\text{slices}}$$

- A variação de altura por *slice* é:

$$\Delta y = \frac{\text{height}}{\text{turns} * \text{slices}}$$

2.5.6.2. Coordenadas dos Pontos

Com base nestes parâmetros, a posição de cada ponto é calculada:

$$x = \text{raio} \cos(\theta)$$

$$y = \text{altura inicial} + n * \Delta y$$

$$x = \text{raio} \sin(\theta)$$

2.5.6.3. Geração da Superfície

A hélice é gerada com uma espécie de “cinta” ou “faixa” a acompanhá-la, criando uma fita fina que representa a espessura da mola. Cada segmento da fita é um pequeno retângulo, dividido em dois triângulos.

Processo

Para cada par de pontos consecutivos ao longo da hélice, são gerados:

- Dois triângulos para a face externa;
- Dois triângulos para a face interna.

Esta técnica garante que a hélice não é apenas uma curva, mas uma estrutura com alguma espessura visual.

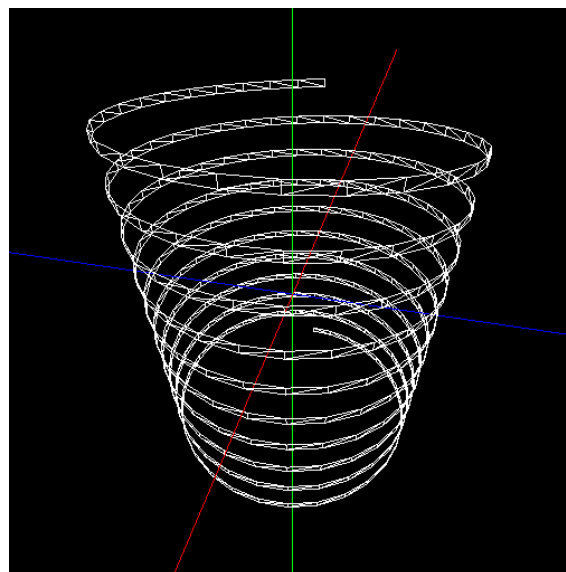


Figura 9: Ilustração de uma hélice com raio 2, altura 5, 10 *turns* e 30 *slices*

3. Engine

O **Engine** desenvolvido permite a visualização das primitivas, através da leitura e interpretação de ficheiros XML com as configurações iniciais.

O ficheiro XML recebido deve estar bem estruturado, contendo as seguintes informações:

- Tamanho da janela (*window*), definido pelos atributos *width* e *height*;
- Parâmetros da câmara (*camera*), incluindo:
 - Posição da câmara (*position*), determinada pelos atributos `x`, `y` e `z`;
 - Ponto de observação (*lookAt*), indicando para onde a câmara está direcionada;
 - Orientação da câmara (*up*), especificando o vetor que representa a direção “para cima”;
 - Projeção (*projection*), definida pelo ângulo de visão (*fov*), distância mínima (*near*) e máxima (*far*).
- Grupo de modelos (*group*), onde cada modelo é representado por uma referência ao ficheiro `.3d` correspondente.

Além das configurações definidas inicialmente com a execução do **Engine**, a aplicação permite atualizar a posição da câmara, bem como aplicar zoom. Também é possível alterar o estilo de visualização da primitiva entre linhas, pontos ou preenchida, além de ativar ou desativar a visualização dos eixos da cena, proporcionando maior flexibilidade na exploração do ambiente 3D.

3.1. Renderização da Cena

3.1.1. Leitura e Interpretação do XML

A configuração do motor gráfico é carregada a partir de um ficheiro XML. Esse processo é realizado pela função `setupConfig`, que recebe o caminho do ficheiro e faz as verificações necessárias para garantir uma correta extração. A função `parseConfig` é utilizada para interpretar o ficheiro, utilizando a biblioteca **RapidXML** para processar os dados e armazená-los em estruturas apropriadas.

3.1.1.1. Configuration

A estrutura principal de armazenamento das configurações extraídas do XML é a classe **Configuration**. Ela contém as dimensões da janela, os parâmetros da câmara e a lista de modelos a serem carregados para renderização.

A função `parseConfig` executa as seguintes etapas:

- **Leitura do ficheiro XML:** O conteúdo do ficheiro é lido e armazenado em uma *string*.
- **Interpretação com RapidXML:** O XML é analisado e os seus nós são percorridos para extrair as configurações necessárias.
- **Armazenamento nas estruturas de dados:** As informações extraídas são utilizadas para inicializar as classes **Window**, **Camera** e a lista de **modelos**.

3.1.1.2. Window

O nó da `window` contém os atributos que definem a resolução da janela de renderização. Os atributos largura (`width`) e comprimento (`height`) são extraídos e convertidos em valores numéricos dentro da classe **Window**.

3.1.1.3. Camera

A configuração da câmera é extraída do nó `camera`, onde são definidas as informações de `position`, `lookAt`, `up`, `fov`, `near` e `far`.

Essas informações são usadas para definir a perspectiva da visualização no **Engine**, garantindo um enquadramento correto da cena 3D.

3.1.2. Renderização dos Modelos

O sistema de renderização de modelos 3D implementado é responsável por carregar ficheiros com modelos tridimensionais, processá-los e desenhá-los no ecrã utilizando a biblioteca OpenGL. Este processo tem em consideração dois formatos principais de ficheiros: `.3d` e `.obj`, cada um tratado de forma diferente, consoante a estrutura do respetivo formato.

3.1.2.1. Leitura de ficheiros e *parsing*

Para os ficheiros `.3d`, o *parsing* é feito pela função `parse3Dfile()`, que lê diretamente uma lista de pontos do ficheiro. A primeira linha contém o número total de pontos e as linhas seguintes contém as coordenadas `x`, `y`, `z` de cada ponto. Cada conjunto de três pontos forma um triângulo, que será posteriormente desenhado.

Para os ficheiros `.obj`, o *parsing* é mais complexo, sendo realizado pela função `parseOBJfile()`. Neste caso, o ficheiro é lido linha a linha, identificando:

- **Vértices**, identificados pela letra `v`, que representam os pontos tridimensionais do modelo.
- **Faces**, identificadas pela letra `f`, que descrevem quais os vértices (indicados por índice) que compõem cada face do objeto. Neste projeto, as faces são todas triângulos, logo cada face tem exatamente 3 vértices.

Como os ficheiros `.obj` não apresentam os vértices diretamente na ordem correta para desenhá-los, a função `parseOBJfile()` reconstrói uma nova lista ordenada de pontos, onde cada grupo de 3 pontos representa um triângulo pronto para ser desenhado.

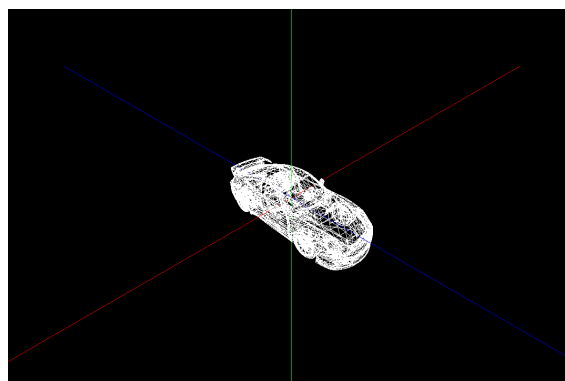


Figura 10: Desenho de um ficheiro porsche.obj

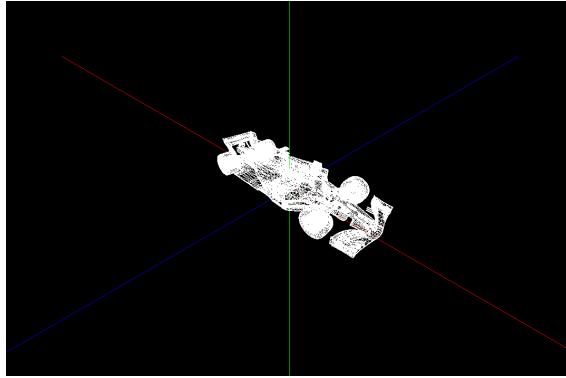


Figura 11: Desenho de um ficheiro ferrari.obj

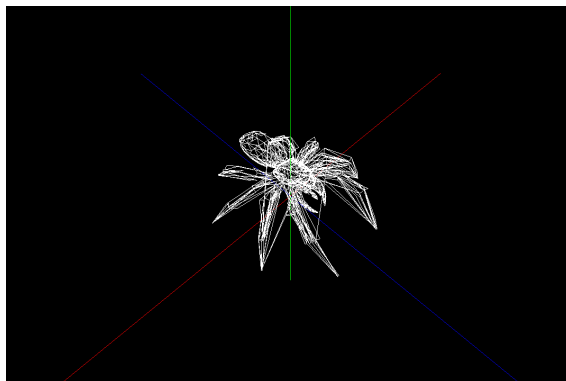


Figura 12: Desenho de um ficheiro spider.obj

A função `parseFile()` atua como ponto de entrada único para a leitura de ficheiros, identificando a extensão do ficheiro e invocando a função de *parsing* correspondente (ou `parse3Dfile()` ou `parseOBJfile()`).

3.1.2.2. Armazenamento e desenho

Após o *parsing*, cada modelo é armazenado numa lista global de modelos (um `std::vector` de listas de pontos). Durante a renderização da cena, cada modelo é desenhado utilizando a função `drawTriangles()`, que percorre a lista de pontos e desenha cada triângulo com chamadas à função `glVertex3f()`. Esta função pertence à biblioteca `OpenGL` e serve para enviar coordenadas de vértices diretamente para a GPU.

3.2. Exploração da Cena

De modo a que o utilizador consiga explorar o objeto 3D gerado, optamos por implementar uma **câmara dinâmica**, permitindo uma observação geral da cena. A câmara pode rodar, aproximar-se e afastar-se do objeto, proporcionando uma experiência interativa ao utilizador.

3.2.1. Movimento Orbital

A câmara segue um **movimento orbital** em torno da origem do cenário, permitindo ao utilizador observar o modelo tridimensional de diferentes ângulos.

Esse movimento simula a rotação de uma câmara sobre uma esfera imaginária centrada na origem, onde a posição da câmara é definida por um raio e dois ângulos de rotação.

O controlo deste movimento é feito através das setas do teclado:

- $\leftarrow$ / $\rightarrow$ ajustam o ângulo horizontal (α), girando a câmara ao redor do eixo Y.
- $\uparrow$ / $\downarrow$ ajustam o ângulo vertical (β), movimentando a câmara para cima ou para baixo

Dessa forma, o utilizador pode orbitar livremente em torno do objeto renderizado sem alterar a sua distância.

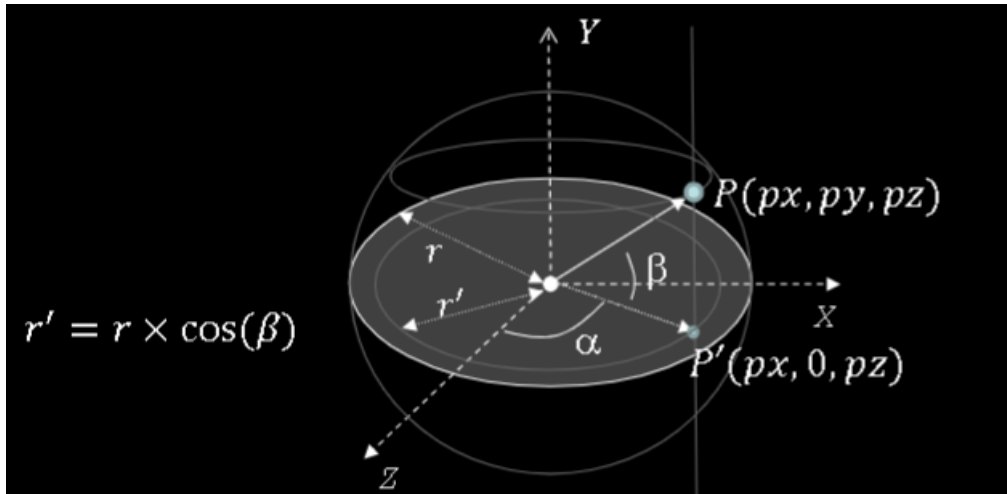


Figura 13: Diagrama do movimento orbital da câmara

A posição exata da câmara é calculada com base nesses ângulos e no raio da esfera imaginária que define a órbita:

```
camX = camRadius * cos(camElevation) * sin(camHorizontal);
camY = camRadius * sin(camElevation);
camZ = camRadius * cos(camElevation) * cos(camHorizontal);
```

Esse cálculo garante que os movimentos orbitais não alterem a distância fixa da origem, proporcionando uma visão panorâmica suave e intuitiva da cena.

3.2.2. Zoom

Para facilitar a visualização de detalhes, a câmara criada apresenta também funcionalidades de *zoom in* e *zoom out*:

- **Tecla 'o'**: Afasta a câmara da origem (*Zoom Out*).
- **Tecla 'i'**: Aproxima a câmara da origem (*Zoom In*).

A distância entre a câmara e a origem é controlada pelo raio da esfera imaginária (`camRadius`), que é, assim, ajustada dinamicamente para aumentar ou diminuir a proximidade.

3.2.3. Outros

Além das funcionalidades de rotação e *zoom*, a câmara possui outras opções adicionais que facilitam a exploração da cena:

- **Tecla 'a'**: Ativa ou desativa os eixos de referência (útil para verificar a orientação dos objetos).
- **Tecla 'r'**: Reinicia a posição da câmara para a configuração inicial.
- **Tecla '1', '2' e '3'**: Alternam entre diferentes modos de renderização (preenchido, *wireframe* e pontos).
- **Tecla 'ESC'**: Encerra o programa.

Estas funcionalidades agilizam e facilitam a exploração da cena, permitindo ao utilizador analisar os objetos renderizados de diversas formas, de acordo com o seu objetivo ou gosto.

4. Utilitários

4.1. Bibliotecas

Para assegurar o correto funcionamento da aplicação, tornou-se fundamental a integração de uma biblioteca específica. Optou-se pela `RapidXML`, uma biblioteca dedicada ao processamento de ficheiros XML, cuja implementação permitiu realizar a leitura dos ficheiros responsáveis por definir as propriedades da câmara e dos modelos a renderizar, proporcionando uma solução eficiente e fiável para essa tarefa em particular.

4.2. Scripts

De forma a facilitar a utilização do projeto, foram desenvolvidos uma série de *scripts* que automatizam processos como compilação, limpeza e testes.

Por enquanto, estes *scripts* apenas funcionam em ambientes Linux e encontram-se na pasta `scripts/`.

Antes de executar qualquer script, é necessário garantir que possuem permissão de execução. Caso contrário, utilize o seguinte comando para conceder permissão: `chmod +x scripts/*`

Após conceder as permissões necessárias, dirija-se ao diretório raiz e execute os comandos pretendidos.

Como *scripts* de compilação, temos:

- `build.sh` – Compila o projeto, gerando os binários na pasta `build/`.
- `clean.sh` – Remove arquivos temporários e binários compilados.
- `rebuild.sh` – Executa a limpeza com `clean.sh` e recompila o projeto do zero com o `build.sh`.

Além disso, foram criados diversos scripts que geram os modelos correspondentes e executam o engine para visualizar a cena. Cada um desses scripts realiza três etapas automaticamente:

1. **Compilação do projeto** (caso ainda não esteja compilado).
2. **Geração do modelo** necessário usando o `generator`.
3. **Execução do engine** com um arquivo XML de teste.

Exemplos de *scripts* de teste:

- `plane1_p1.sh` - Gera um modelo de um **plano** de tamanho 4 e com 6 divisões, exibindo-o.
- `helix1_p1.sh` - Gera um modelo de uma **espiral** com 2 de raio, 5 de altura, 10 voltas e 30 fatias, exibindo-o.
- `torus1_p1.sh` - Gera um modelo de um **toro** com o raio maior igual a 3, com a diferença de raios igual a 1, 20 lados e 15 círculos, exibindo-o.
- `testx_p1.sh` - 5 *scripts* ($x \in [1, \dots, 5]$) que geram os modelos de um **cone** (teste 1 e 2), uma **esfera** (teste 3), uma **caixa** (teste 4) e uma **esfera** em conjunto com um **plano** (teste 5), exibindo-os.
- `ferrari_p1.sh`, `porsche_p1.sh` e `spider_p1.sh` - Exibem os modelos **.obj** correspondentes.

Esses *scripts* garantem um fluxo de trabalho mais ágil, eliminando a necessidade de executar os comandos manualmente.

4.3. Classes Comuns

Com o objetivo de tornar o projeto mais modular e adaptável, desenvolveu-se a classe `Point`, responsável por representar cada ponto no espaço tridimensional (x , y , z). Assim, cada primitiva é descrita através de um vetor composto por objetos `Point`.

5. Conclusões e Trabalho Futuro

A implementação deste projeto permitiu-nos consolidar conhecimentos fundamentais de `C++` e `OpenGL`, aplicando-os diretamente na geração de modelos geométricos simples, como caixas, esferas e cones. Este trabalho inicial serviu como uma base sólida para as fases futuras do desenvolvimento do nosso projeto.